

UNIVERSIDADE FEDERAL DA PARAÍBA
CENTRO DE INFORMÁTICA

Disciplina: Aprendizagem de Máquina — 2026.1
Professores: Bruno Jefferson de Sousa Pessoa e Gilberto Farias de Sousa Filho

MANUTENÇÃO PREDITIVA INDUSTRIAL

*Previsão de Falhas em Máquinas a partir de Leituras de Sensores com Redes Neurais
e Árvores de Decisão*

Relatório do Projeto Final da Disciplina
Integrantes: Anne Fernandes da Costa Oliveira e João Vitor Pereira da Costa

João Pessoa — PB
Agosto de 2026

Sumário

- 1. Introdução3
- 2. Dataset3
- 3. Tratamento de Dados6
- 4. Rede Neural (Perceptron Multicamadas)8
- 5. Árvore de Decisão e Random Forest13
- 6. Comparação e Escolha do Melhor Modelo19
- 7. Conclusão21

1. Introdução

A manutenção preditiva é uma abordagem de gestão de ativos industriais que utiliza dados de sensoriamento operacional para antecipar falhas em equipamentos antes que elas ocorram, em contraste com a manutenção corretiva (reagir após a falha) e a manutenção preventiva (baseada em cronogramas fixos, independentemente da real condição da máquina). No contexto da Indústria 4.0, a disponibilidade de sensores de baixo custo e a capacidade de processar grandes volumes de dados operacionais tornaram viável a aplicação de técnicas de Aprendizagem de Máquina para prever falhas com base no estado real de cada equipamento.

Este projeto tem como objetivo aplicar os conceitos de Aprendizagem de Máquina (AM) estudados na disciplina — especificamente Redes Neurais Artificiais e Árvores de Decisão — a um problema prático de classificação binária: prever se uma máquina industrial irá falhar (Machine failure = 1) ou operar normalmente (Machine failure = 0) a partir de cinco variáveis contínuas capturadas por sensores (temperatura do ar, temperatura do processo, velocidade rotacional, torque e desgaste da ferramenta de corte) e de uma variável categórica que identifica a qualidade do produto fabricado (Type: L, M ou H).

A relevância prática do problema é direta: falhas não detectadas em máquinas industriais podem causar paradas de produção não planejadas, com custos financeiros elevados e, em certos contextos, risco à segurança dos operadores. Um sistema de classificação capaz de sinalizar antecipadamente o risco de falha permite à equipe de manutenção agir de forma proativa, otimizando o uso de recursos e reduzindo o tempo de máquina parada (downtime).

O relatório está organizado da seguinte forma: a Seção 2 descreve a fonte e a composição do dataset utilizado; a Seção 3 detalha o tratamento de dados realizado, incluindo a construção da matriz de entrada e do vetor-alvo; as Seções 4 e 5 apresentam, respectivamente, a implementação e os resultados dos modelos de Rede Neural e de Árvore de Decisão (com uma extensão em Random Forest); a Seção 6 compara os modelos e justifica a escolha do modelo final; e a Seção 7 apresenta as conclusões e limitações do trabalho.

2. Dataset

Fonte: AI4I 2020 Predictive Maintenance Dataset, disponibilizado publicamente pelo UCI Machine Learning Repository (Matzka, 2020). O dataset foi sinteticamente gerado para refletir, de forma realista, padrões de falha observados em processos industriais reais, sendo amplamente utilizado como benchmark acadêmico para problemas de manutenção preditiva.

O conjunto de dados contém 10.000 registros de operação de máquinas industriais, cada um descrito por 14 colunas. As variáveis de interesse para a modelagem são:

Variável	Descrição	Tipo
Type	Qualidade do produto fabricado: L (Low), M (Medium) ou H (High)	Categórica
Air temperature [K]	Temperatura do ar ao redor da máquina	Numérica
Process temperature [K]	Temperatura do processo produtivo	Numérica

Variável	Descrição	Tipo
Rotational speed [rpm]	Velocidade rotacional do eixo	Numérica
Torque [Nm]	Torque aplicado ao processo	Numérica
Tool wear [min]	Tempo acumulado de desgaste da ferramenta	Numérica
Machine failure	Alvo: 1 = falha, 0 = operação normal	Binária (alvo)

Colunas excluídas da modelagem: UDI e Product ID são identificadores sem valor preditivo. As colunas TWF, HDF, PWF, OSF e RNF representam os submodos específicos de falha que compõem o rótulo Machine failure — usá-las como features causaria vazamento de dados (data leakage), pois são, na prática, uma decomposição do próprio alvo.

2.1 Validação de Consistência do Rótulo

Antes de iniciar a modelagem, verificou-se a consistência entre o rótulo Machine failure e os cinco submodos de falha que teoricamente o compõem. Dos 10.000 registros, 339 apresentam Machine failure = 1, enquanto 348 apresentam pelo menos um submodo de falha ativo. Essa pequena discrepância resultou em 9.973 linhas consistentes e 27 linhas inconsistentes (0,27% do total) — em sua maioria, casos de RNF (Random Failure, um submodo estocástico por definição, que não necessariamente implica falha classificada) sem correspondência exata em Machine failure. Dado o volume marginal, essas inconsistências foram mantidas no dataset sem tratamento especial, não afetando a integridade da modelagem.

2.2 Desbalanceamento de Classes

O dataset apresenta desbalanceamento severo entre as classes: apenas 339 dos 10.000 registros (3,39%) correspondem a falhas reais. Essa característica foi um fator determinante em praticamente todas as decisões metodológicas subsequentes — da escolha da métrica de avaliação ao tratamento de hiperparâmetros dos modelos — e é discutida em detalhe ao longo deste relatório.

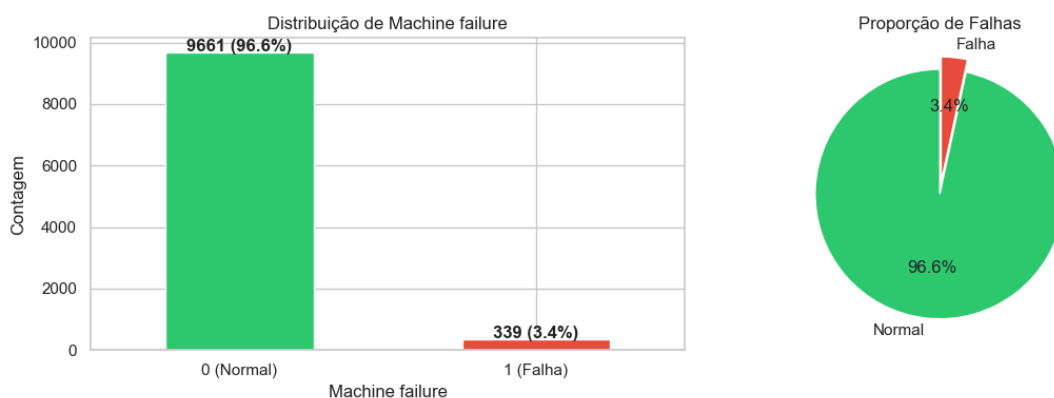


Figura 1 — Distribuição da variável alvo Machine failure. Apenas 3,39% dos registros correspondem a falhas.

2.3 Análise Exploratória de Dados (EDA)

Para compreender melhor o comportamento das variáveis antes da modelagem, realizou-se uma análise exploratória cobrindo a distribuição dos submodos de falha, a relação entre o tipo de produto e a taxa de falha, e a separabilidade das classes em cada variável numérica.

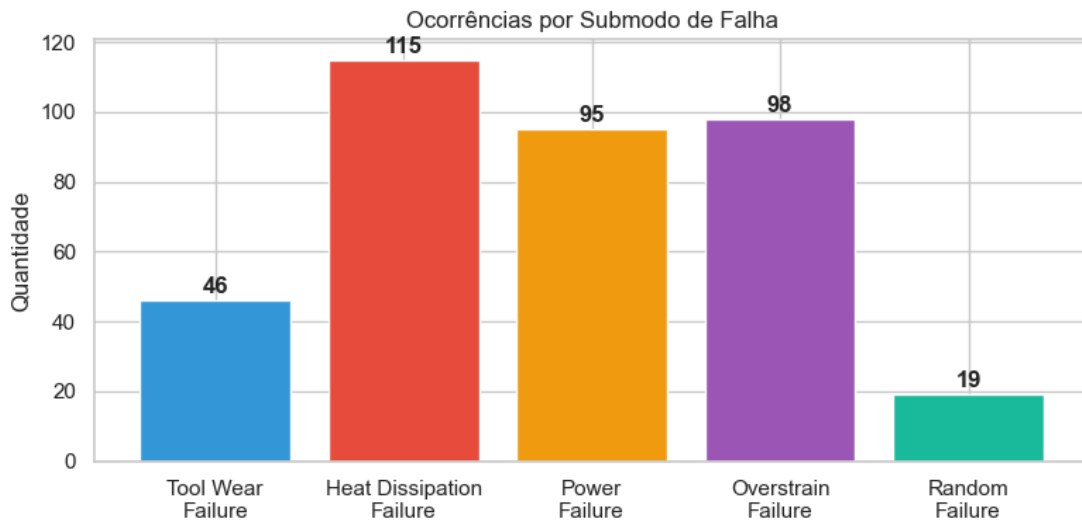


Figura 2 — Ocorrências por submodo de falha (TWF, HDF, PWF, OSF, RNF).

A variável Type também apresenta relação com a taxa de falha: produtos de qualidade L (Low) apresentam taxa de falha de 3,92%, contra 2,77% para M (Medium) e 2,09% para H (High) — um padrão coerente com a expectativa de que peças de menor qualidade estejam sujeitas a maior variabilidade de tolerância mecânica.

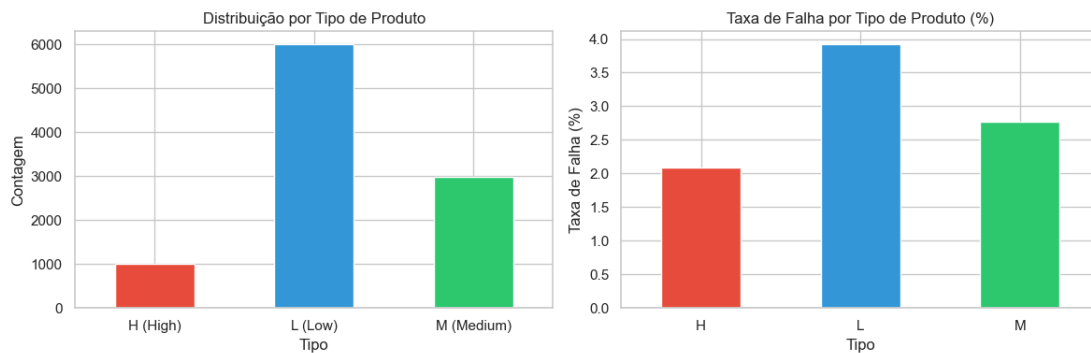


Figura 3 — Distribuição por tipo de produto e taxa de falha por tipo.

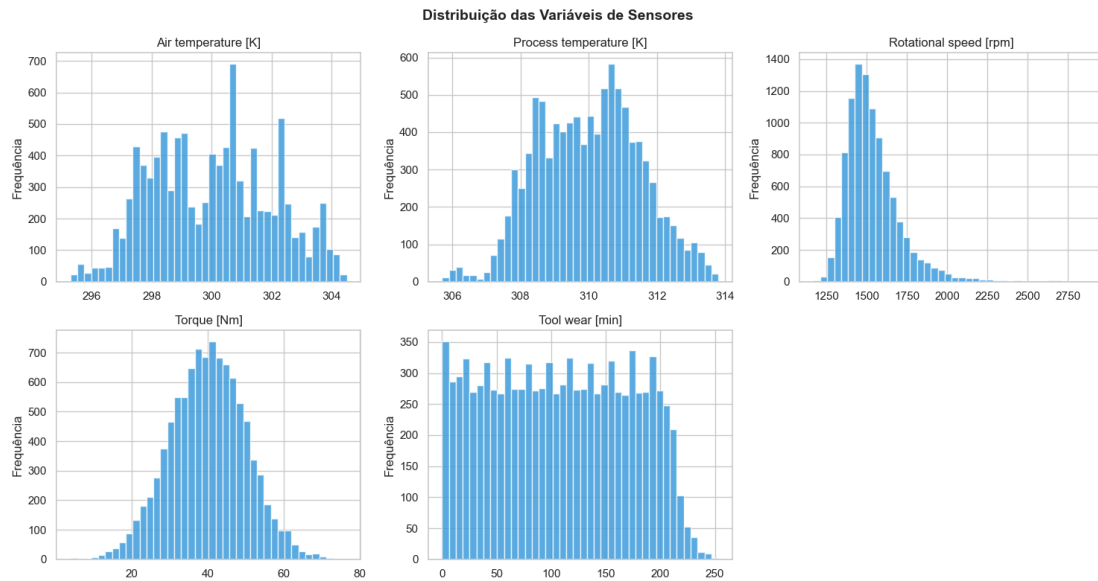


Figura 4 — Distribuição das cinco variáveis numéricas de sensores.

A comparação das distribuições entre registros normais e registros de falha (Figura 5) revela indícios de separabilidade em algumas variáveis, especialmente Torque e Tool wear, cujas distribuições para a classe de falha se deslocam visivelmente em relação à classe normal — um sinal favorável para a capacidade preditiva dos modelos.

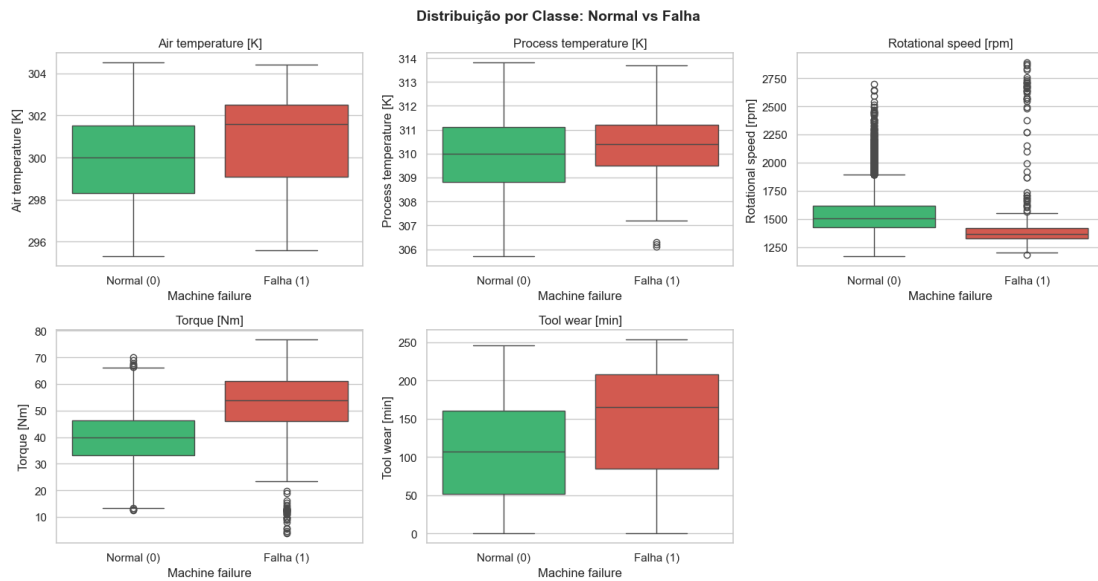


Figura 5 — Comparação das distribuições por classe (Normal vs. Falha) para cada variável numérica.

3. Tratamento de Dados

3.1 Seleção de Features e Análise de Correlação

Calculou-se a correlação de Pearson entre cada variável numérica candidata e o alvo Machine failure, com o objetivo de identificar efeitos individuais relevantes:

Feature	Correlação com Machine failure
Torque [Nm]	+0,1913
Tool wear [min]	+0,1054
Air temperature [K]	+0,0826
Rotational speed [rpm]	-0,0442
Process temperature [K]	+0,0359

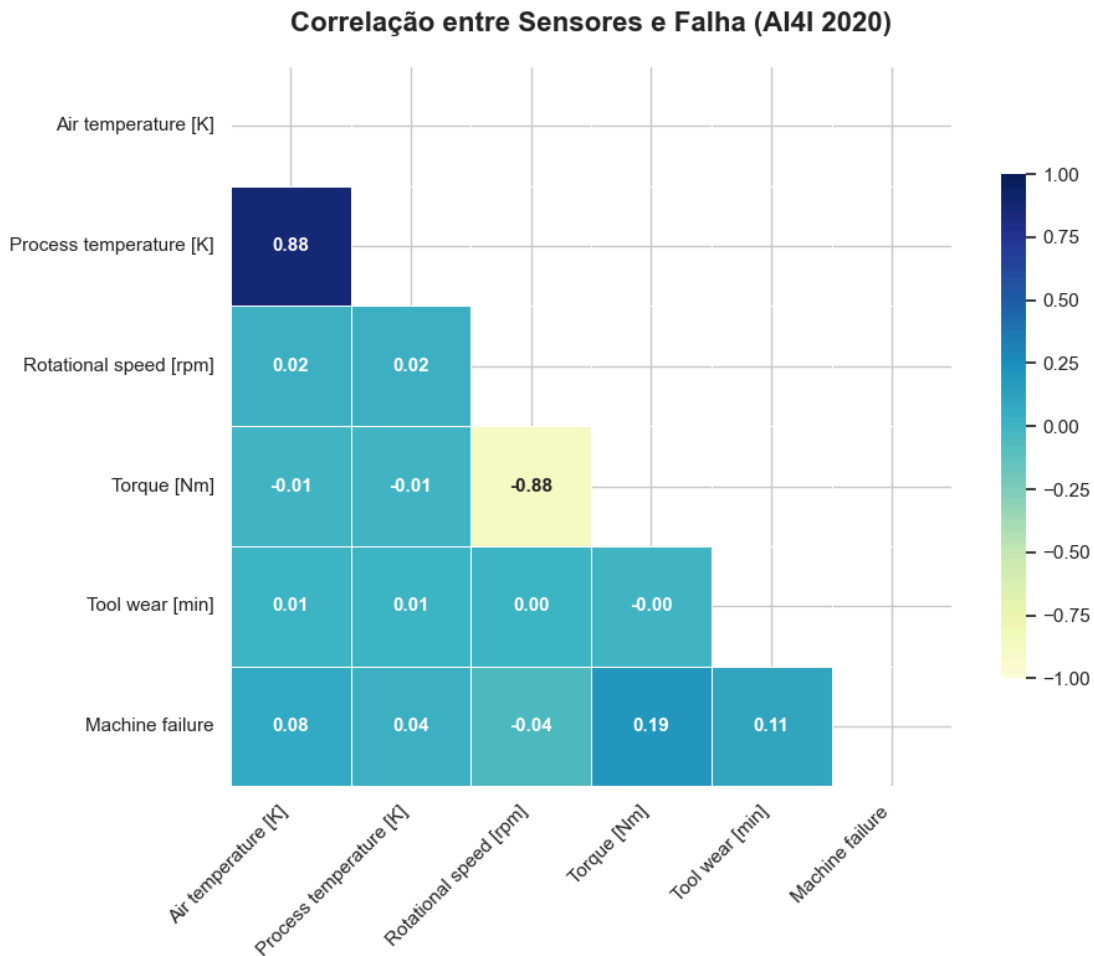


Figura 6 — Matriz de correlação entre as variáveis de sensores e o alvo Machine failure.

Optou-se por NÃO aplicar um filtro de exclusão por limiar de correlação individual neste dataset. Isso porque (i) há apenas 5 variáveis numéricas candidatas — um conjunto já reduzido, e (ii) os modos de falha reais do AI4I 2020 são definidos por REGRAS DE COMBINAÇÃO entre sensores, não por efeitos lineares isolados. Por exemplo, a falha por dissipação de calor (HDF) depende simultaneamente da diferença entre a temperatura do processo e a temperatura do ar ser inferior a 8,6 K E da velocidade rotacional ser inferior a 1380 rpm; a falha de potência (PWF) depende do produto Torque $\times$ Velocidade Rotacional (a potência mecânica) estar fora de uma faixa aceitável. Uma variável pode, portanto, ter correlação individual baixa com o alvo e ainda ser altamente relevante em combinação com outra — motivo pelo qual as cinco variáveis numéricas foram mantidas integralmente como features.

3.2 Codificação de Variáveis Categóricas

A variável Type (L, M, H) foi convertida por one-hot encoding. Para evitar a dummy variable trap — redundância perfeita entre as colunas geradas a partir de uma variável categórica de k categorias —, removeu-se a categoria de referência (H), resultando nas colunas binárias Type_L e Type_M.

3.3 Composição Final das Features (X)

O conjunto final de 7 features utilizado em todos os modelos é:

#	Feature
1	Torque [Nm]
2	Tool wear [min]
3	Air temperature [K]
4	Rotational speed [rpm]
5	Process temperature [K]
6	Type_L
7	Type_M

Não foram encontrados valores faltantes em nenhuma das features selecionadas.

3.4 Divisão Treino/Teste Estratificada

Dado o desbalanceamento severo (~3,4% de falhas), a divisão treino/teste foi realizada com o parâmetro stratify=y, garantindo que a proporção original de classes fosse preservada em ambos os subconjuntos. Sem essa precaução, uma divisão puramente aleatória poderia resultar em um conjunto de teste com proporção de falhas muito distinta da original, ou até mesmo sem nenhum exemplo positivo, invalidando a avaliação dos modelos. Reservou-se 20% dos dados para teste (proporção padrão) e 80% para treino.

Conjunto	Amostras	Normais (0)	Falhas (1)	Proporção de falhas
Treino	8.000	7.729	271	3,4%
Teste	2.000	1.932	68	3,4%

3.5 Normalização

As variáveis numéricas possuem escalas muito distintas (temperatura em Kelvin ≈ 300 , velocidade em rpm ≈ 1500 , torque em Nm ≈ 40), o que prejudicaria o treinamento da Rede Neural caso não fossem padronizadas. Aplicou-se o StandardScaler do scikit-learn, ajustado (fit) exclusivamente no conjunto de treino e aplicado (transform) ao conjunto de teste, evitando qualquer vazamento de informação do teste para o treino.

Dimensões finais: N = 8.000 amostras de treino, p = 7 features. Essas duas quantidades são a base para o cálculo da dimensão VC da Rede Neural, apresentado na Seção 4.

4. Rede Neural (Perceptron Multicamadas)

Nesta seção, constrói-se e treina-se uma Rede Neural Artificial (MLP — Multi-Layer Perceptron) para prever Machine failure, seguindo a teoria de generalização estudada na disciplina: dimensionamento da arquitetura pela Dimensão VC, separação de conjunto de validação, busca em grade de hiperparâmetros, tratamento do desbalanceamento de classes, análise de overfitting e calibração do limiar de decisão.

4.1 Definição da Arquitetura pela Teoria da Generalização

4.1.1 Dimensão VC e Número de Pesos

A Teoria Estatística do Aprendizado de Vapnik-Chervonenkis (VC) estabelece que a capacidade de um modelo é proporcional à sua Dimensão VC (d_{VC}). Em Redes Neurais, a Dimensão VC pode ser aproximada pelo número total de pesos e vieses (bias) da rede: $d_{VC} \approx |W|$.

4.1.2 Regra de Ouro da Generalização

Para evitar overfitting (memorização do treino em vez de aprendizado do padrão real), a Regra de Ouro da Generalização estabelece que é necessário haver, no mínimo, 10 exemplos de treino independentes para cada parâmetro ajustável da rede:

$$N \geq 10 \cdot d_{VC} \Rightarrow N \geq 10 \cdot |W|$$

4.1.3 Escolha de Uma Camada Escondida

Optou-se por uma arquitetura com uma única camada escondida, fundamentada no Teorema da Aproximação Universal (Cybenko, 1989; Hornik, 1991): uma rede feedforward com uma única camada escondida contendo um número finito de neurônios já é capaz de aproximar qualquer função contínua em um subconjunto compacto de $\mathbb{R}^n$. Adicionar mais camadas aumentaria a Dimensão VC sem necessidade teórica, dificultando a generalização dado o tamanho do dataset disponível.

4.1.4 Cálculo do Número Máximo de Neurônios

Para uma arquitetura de uma camada escondida com d entradas, n neurônios escondidos e 1 saída, o número total de pesos é:

$$|W| = (d + 1) \cdot n + (n + 1)$$

Substituindo na Regra de Ouro e isolando n , obtém-se:

$$n \leq (N - 10) / (10(d + 2))$$

4.1.5 Substituição com os Valores Reais

Para este problema, $N = 8.000$ e $d = 7$. Substituindo:

$$n \leq (8000 - 10) / (10 \times (7 + 2)) = 7990 / 90 \approx 88,77$$

Arredondando para baixo, por segurança: $n = 88$ neurônios na camada escondida.

4.1.6 Verificação do Número de Pesos

Com $d = 7$ e $n = 88$:

$$|W| = (7 + 1) \times 88 + (88 + 1) = 704 + 89 = 793 \text{ pesos totais}$$

$$N = 8.000 \geq 10 \times 793 = 7.930 \rightarrow \text{Regra de Ouro satisfeita.}$$

4.1.7 Arquitetura Final

Camada	Neurônios	Ativação	Parâmetros
Entrada	7	—	0
Escondida	88	ReLU	$(7 \times 88) + 88 = 704$
Saída	1	Sigmoid	$(88 \times 1) + 1 = 89$
Total	—	—	793

4.2 Divisão em Treino e Validação

Conforme a Regra de Ouro para o tamanho do conjunto de validação, $K \approx N/5$ — aproximadamente 20% do treino reservado para validação, balanceando a necessidade de uma estimativa de erro confiável (E_{val}) com a de não reduzir excessivamente o volume de treino disponível.

O uso de stratify=y_train foi essencial nesta etapa: uma divisão puramente aleatória poderia resultar em um conjunto de validação sem nenhuma amostra da classe minoritária (falha), impossibilitando avaliar a real capacidade do modelo de detectá-la.

A divisão resultou em 6.400 amostras de treino final e 1.600 de validação, ambas mantendo a proporção de 3,4% de falhas.

4.3 Busca em Grade (Grid Search) e Tratamento do Desbalanceamento

4.3.1 Regularização L2

A regularização L2 adiciona uma penalidade proporcional ao quadrado dos pesos à função de custo: $E_{reg} = E_{in} + \lambda \sum w_i^2$. Embora não altere a Dimensão VC teórica do modelo, essa penalidade reduz a Dimensão VC efetiva ao restringir o espaço de hipóteses alcançável durante o treino, ajudando a prevenir overfitting.

4.3.2 O Conceito de class_weight

Como apenas ~3,4% das amostras representam falhas, um modelo sem qualquer ajuste na função de custo tende a aprender a sempre prever a classe majoritária ("não falha"), obtendo, ainda assim, alta acurácia aparente (~96,6%) — o que torna a acurácia bruta uma métrica enganosa para este problema. A opção `class_weight='balanced'` ajusta a função de custo para penalizar proporcionalmente mais os erros cometidos na classe minoritária, forçando a rede a de fato aprender a distingui-la.

4.3.3 Critério de Seleção: F2-Score

Em manutenção preditiva, o custo de um falso negativo (deixar de prever uma falha real, que pode causar parada de produção não planejada) é tipicamente muito maior que o de um falso positivo (enviar a equipe de manutenção verificar uma máquina saudável, gerando apenas um custo de inspeção). Por essa razão, priorizou-se Recall sobre Precisão. O F1-Score pondera as duas métricas

igualmente (50/50), o que não reflete essa assimetria de custos; optou-se, portanto, pelo F2-Score, que pondera o Recall com peso aproximadamente duas vezes maior que a Precisão:

$$F\beta = (1 + \beta^2) \cdot (\text{precisão} \times \text{recall}) / (\beta^2 \times \text{precisão} + \text{recall}), \text{ com } \beta = 2$$

O grid search testou 36 combinações de hiperparâmetros: taxa de aprendizado $\eta \in \{0,001; 0,01\}$, batch size $\in \{16, 32, 64\}$, regularização L2 $\lambda \in \{0; 0,001; 0,01\}$ e class_weight $\in \{\text{Não}, \text{Balanceado}\}$, avaliando cada combinação pelo F2-Score no conjunto de validação.

4.4 Análise de Overfitting (Ein vs. Eout)

Com a melhor combinação de hiperparâmetros, o modelo foi treinado por 150 épocas para observar o comportamento de longo prazo das curvas de perda de treino (Ein) e de validação (Eout).

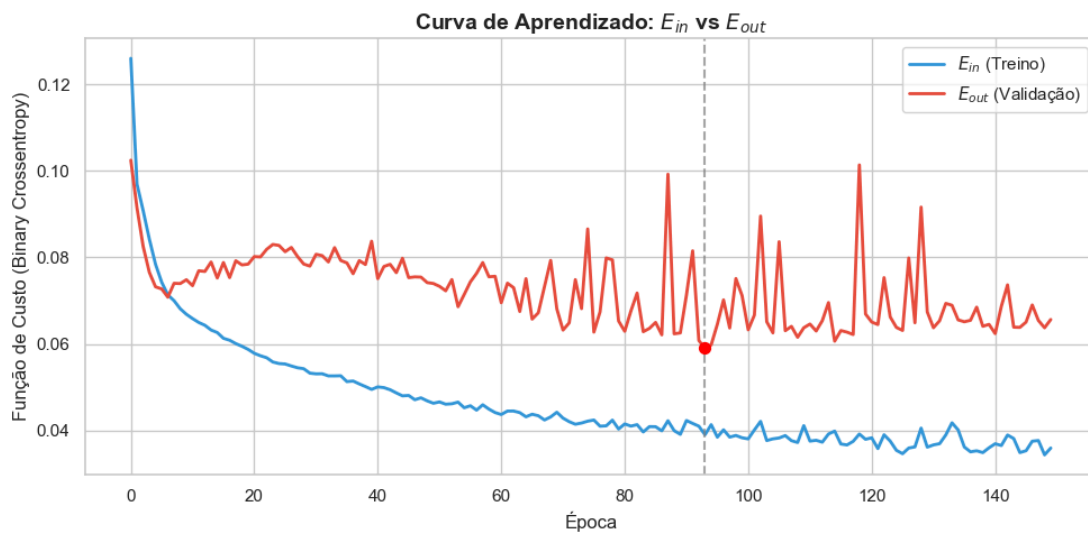


Figura 7 — Curva de aprendizado: perda de treino (E_{in}) vs. perda de validação (E_{out}) ao longo das épocas.

A curva evidencia o comportamento clássico de overfitting: a perda de validação atinge seu ponto mínimo na época 93 (destacado no gráfico) e, a partir daí, passa a oscilar em patamar mais alto, enquanto a perda de treino continua caindo monotonicamente. Isso justifica o uso de EarlyStopping no treinamento do modelo final, interrompendo o treino no momento ótimo e restaurando os pesos da melhor época — evitando a necessidade de escolher manualmente um número fixo de épocas.

4.5 Treinamento do Modelo Final

O modelo final foi treinado com EarlyStopping (monitor='val_loss', patience=15, restore_best_weights=True), interrompendo o treinamento na época 60 e restaurando os pesos correspondentes ao menor erro de validação observado.

4.6 Avaliação no Conjunto de Teste

Dado o desbalanceamento severo, a acurácia isoladamente é enganosa: um modelo que sempre prevê "não falha" já acertaria 96,60% das vezes (baseline). As métricas relevantes para avaliar a capacidade real de detecção são Precisão, Recall, F1 e F2 da classe "Falhou (1)".

Métrica	Baseline	Rede Neural (limiar = 0,50)
Acurácia	0,9660	0,9780
Precisão (classe 1)	—	0,8000
Recall (classe 1)	0,0000	0,4706
F1-Score (classe 1)	—	0,5926
F2-Score (classe 1)	—	0,5128

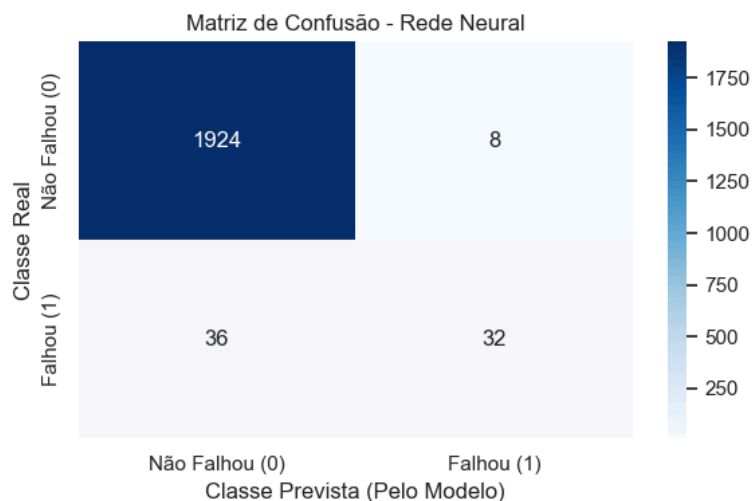


Figura 8 — Matriz de confusão da Rede Neural com limiar padrão (0,50), no conjunto de teste.

4.7 Calibração do Limiar de Decisão

O limiar padrão de 0,5 é uma convenção arbitrária, não uma escolha otimizada para este problema específico. Como o custo de um falso negativo é maior que o de um falso positivo, testaram-se limiares de decisão entre 0,05 e 0,95 (em passos de 0,05), calculando Precisão, Recall, F1 e F2 para cada valor.

Ponto metodológico importante: a busca pelo limiar ótimo é realizada no conjunto de VALIDAÇÃO (X_{val}/y_{val} — o mesmo já utilizado na seleção de hiperparâmetros do grid search), e não no conjunto de teste. Isso preserva o princípio fundamental de que o conjunto de teste deve ser usado uma única vez, apenas na avaliação final — usá-lo também para escolher o limiar contaminaria a estimativa de desempenho, tornando-a artificialmente otimista (uma forma de vazamento de dados).

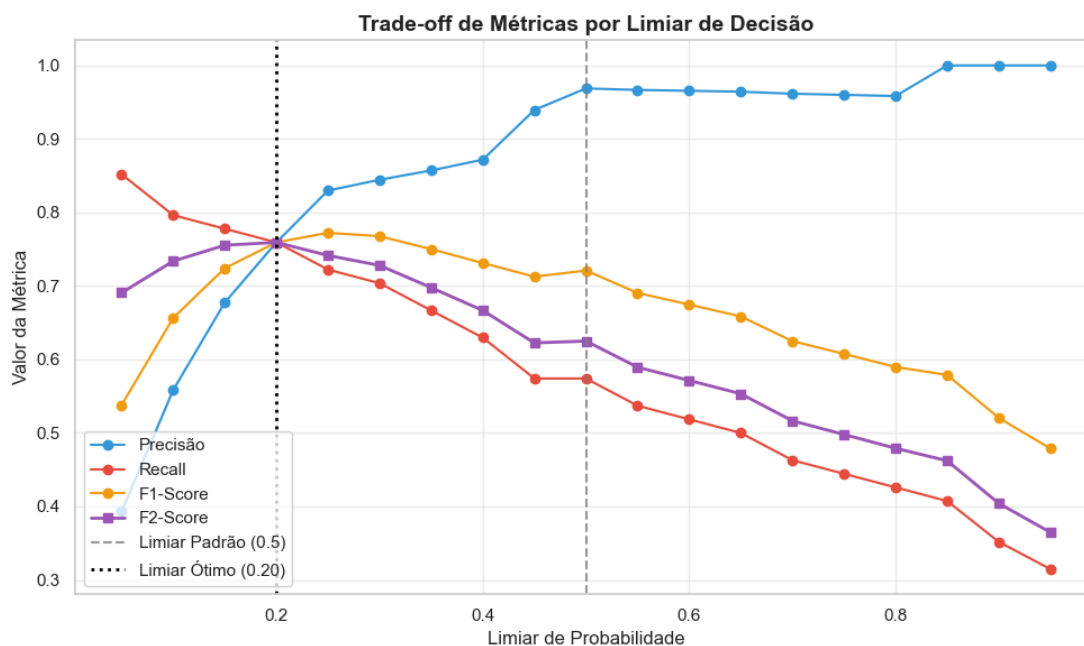


Figura 9 — Trade-off entre Precisão, Recall, F1 e F2 em função do limiar de decisão, calculado no conjunto de validação.

O limiar ótimo encontrado no conjunto de validação foi 0,20 (F2 de validação = 0,7593), valor que passa a ser adotado de forma fixa para a avaliação final.

4.7.1 Avaliação Final no Teste (limiar já fixado)

Com o limiar ótimo já determinado exclusivamente a partir da validação, ele é aplicado uma única vez ao conjunto de teste — esta é a única etapa em que o conjunto de teste é utilizado nesta análise de limiar, e ele não influenciou a escolha do valor, apenas confirma o ganho de desempenho já esperado.

Métrica	Limiar Padrão (0,50)	Limiar Ótimo (0,20)
Recall (classe 1)	0,4706	0,6176
Precisão (classe 1)	0,8000	0,6269
F1-Score (classe 1)	0,5926	0,6222
F2-Score (classe 1)	0,5128	0,6195

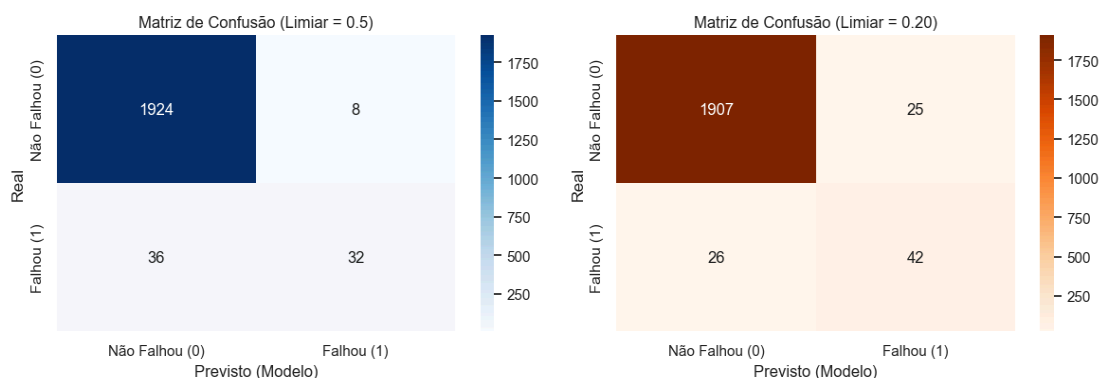


Figura 10 — Comparação das matrizes de confusão no teste: limiar padrão (0,50) vs. limiar ótimo calibrado na validação (0,20).

4.8 Resumo Final da Rede Neural

Item	Valor
Arquitetura	7 → 88 → 1 (uma camada escondida)
Total de pesos W	793
Regra de Ouro	$N = 8.000 \geq 10 \times 793 = 7.930$ ✓
Taxa de aprendizado (η)	0,01
Batch size	16
Regularização L2 (λ)	0,0
Class weight balanceado	Não
Época final (EarlyStopping)	60
Limiar de decisão adotado	0,20 (calibrado na validação)
F2-Score final (teste)	0,6195
Recall final (teste)	0,6176
Precisão final (teste)	0,6269
Acurácia final (teste)	0,9745

5. Árvore de Decisão e Random Forest

Nesta seção, constrói-se e avalia-se um modelo de Árvore de Decisão com poda por Minimal Cost-Complexity Pruning e validação cruzada estratificada, seguido de uma extensão em Random Forest para comparação, mantendo o mesmo critério de seleção de hiperparâmetros (F2-Score) adotado na Rede Neural.

5.1 Árvore de Decisão Sem Poda (Linha de Base)

Treinou-se inicialmente uma árvore sem limite de profundidade nem poda (DecisionTreeClassifier(random_state=42)), que cresce até que cada folha contenha amostras de uma única classe. Essa árvore atingiu profundidade 19 e 152 folhas.

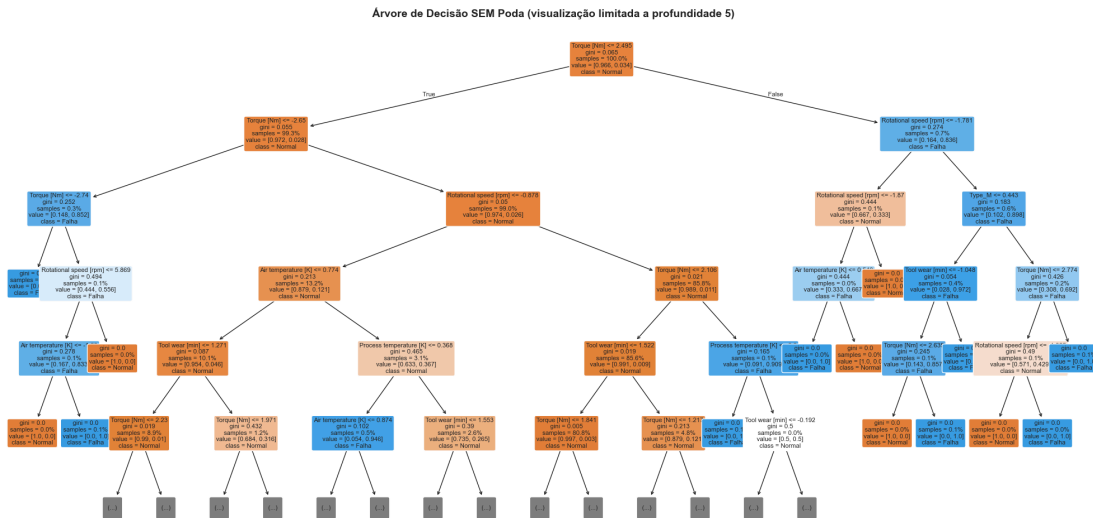


Figura 11 — Árvore de decisão sem poda (visualização cortada em profundidade 5 para legibilidade).

5.2 Análise de Overfitting

Métrica	Treino	Teste
Acurácia	1,0000	0,9770
Erro (E)	0,0000	0,0230
F2-Score	1,0000	0,6509

O erro de treino igual a zero ($E_{in} \approx 0$) evidencia overfitting: a árvore memorizou completamente o conjunto de treino, sem capacidade de generalização otimizada. Essa observação motiva o processo de poda apresentado a seguir.

5.3 Poda via Minimal Cost-Complexity Pruning

5.3.1 O Conceito

A poda por custo-complexidade busca minimizar, para cada sub-árvore T , o custo total:

$$R\alpha(T) = R(T) + \alpha \cdot |\tilde{T}|$$

onde $R(T)$ é a impureza total (soma ponderada da impureza de Gini em cada folha), $|\tilde{T}|$ é o número de folhas terminais e $\alpha \geq 0$ é o parâmetro de complexidade (`ccp_alpha` no `scikit-learn`). Quando $\alpha = 0$, a árvore cresce sem restrição; conforme α aumenta, a penalidade por folha adicional cresce, forçando a poda de ramos que não compensam a melhoria de pureza obtida.

5.3.2 Seleção do α Ótimo por Validação Cruzada Estratificada

Seguindo a mesma Regra de Ouro aplicada à Rede Neural ($K \approx N/5$), utilizou-se validação cruzada estratificada com 5 folds (StratifiedKFold), preservando a proporção de ~3,4% de falhas em cada fold — fundamental para uma avaliação confiável em dados desbalanceados. Testaram-se 64 valores candidatos de α no intervalo $[0; 0,008916]$, obtidos via `cost_complexity_pruning_path`, cada um avaliado com e sem `class_weight='balanced'`, usando F2-Score como métrica.

Cenário	α ótimo	F2 (validação cruzada)
Sem class_weight	0,000317	0,7010 $\pm$ 0,0674
Com class_weight = balanced	0,000834	0,6790 $\pm$ 0,0395

O cenário sem class_weight obteve o maior F2 médio e foi o escolhido: $\alpha = 0,000317$.

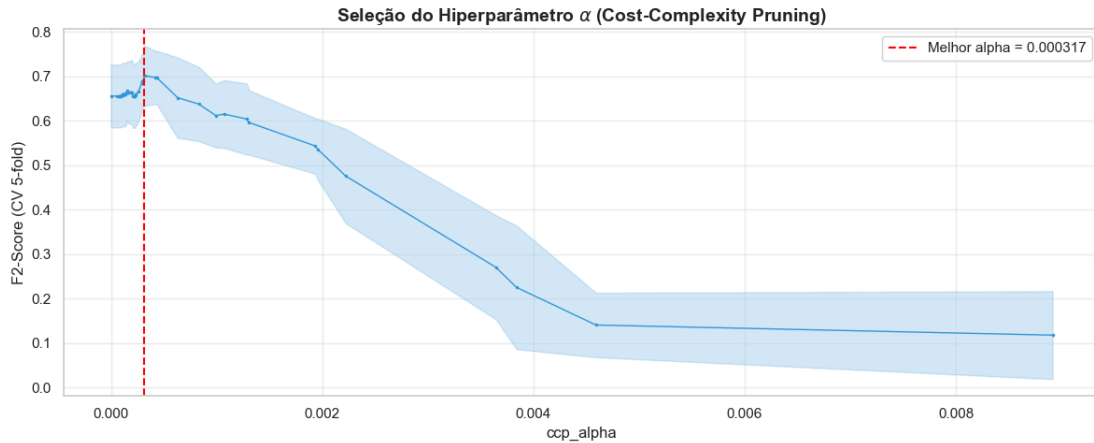


Figura 12 — F2-Score médio (validação cruzada 5-fold) em função de α , com o α ótimo destacado.

5.4 Árvore Final Podada

A poda reduziu drasticamente a complexidade da árvore:

Métrica estrutural	Sem poda	Podada ($\alpha = 0,000317$)
Profundidade	19	11
Número de folhas	152	30

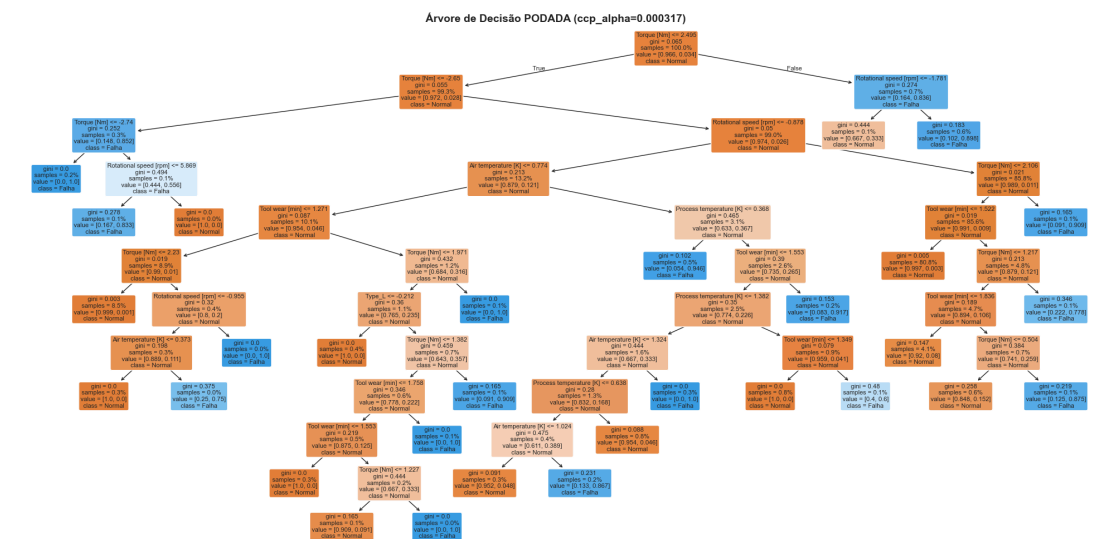


Figura 13 — Árvore de decisão final, após poda por custo-complexidade.

5.5 Avaliação no Conjunto de Teste

Métrica	Baseline	Árvore Podada
Acurácia	0,9660	0,9855
Precisão (classe 1)	—	0,8000
Recall (classe 1)	0,0000	0,7647
F1-Score (classe 1)	—	0,7820
F2-Score (classe 1)	—	0,7715

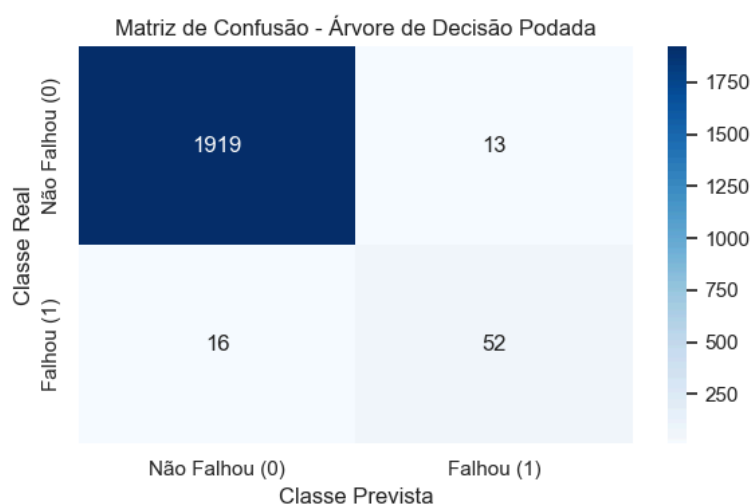


Figura 14 — Matriz de confusão da Árvore de Decisão Podada.

5.6 Importância das Features

O atributo `feature_importances_` (baseado na redução de impureza de Gini) revela a contribuição relativa de cada variável para as decisões da árvore:

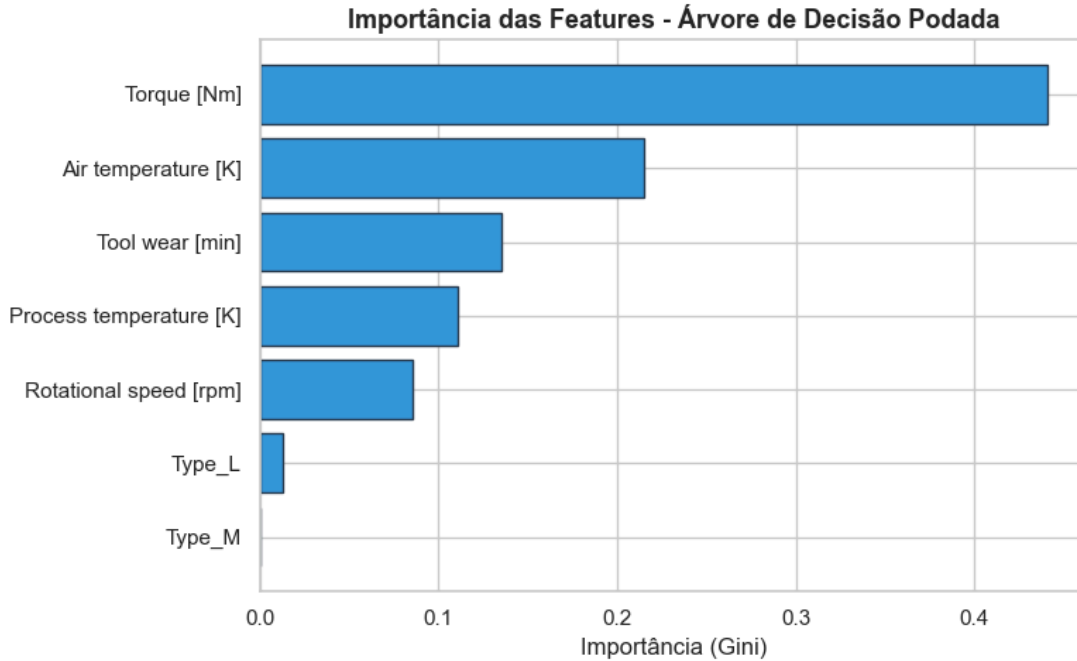


Figura 15 — Importância das features na Árvore de Decisão Podada.

Feature	Importância (Gini)
Torque [Nm]	0,4409
Air temperature [K]	0,2148
Tool wear [min]	0,1355
Process temperature [K]	0,1107
Rotational speed [rpm]	0,0854
Type_L	0,0127
Type_M	0,0000

O resultado confirma que Torque, Tool wear e as variáveis de temperatura — exatamente as variáveis que compõem as regras físicas reais de falha do dataset (potência mecânica, desgaste da ferramenta e dissipação térmica) — são as mais relevantes, evidenciando que o modelo capturou sinal real, e não ruído.

5.7 Random Forest (Extensão)

Como extensão complementar à Árvore de Decisão, treinou-se também um Random Forest — um ensemble de múltiplas árvores treinadas em subamostras diferentes dos dados, cuja predição final é obtida por votação majoritária. O grid search testou 12 combinações ($n_estimators \in \{100, 200\}$, $max_depth \in \{\text{Nenhum}, 10, 20\}$, $class_weight \in \{\text{Nenhum}, \text{balanced}\}$), com a mesma validação cruzada 5-fold estratificada e F2-Score como critério.

Melhor combinação: $n_estimators = 100$, $max_depth = 10$, $class_weight = \text{balanced}$ (F2 de validação cruzada = $0,5748 \pm 0,0572$).

Métrica	Random Forest
Acurácia	0,9760
Precisão (classe 1)	0,6613
Recall (classe 1)	0,6029
F1-Score (classe 1)	0,6308
F2-Score (classe 1)	0,6138

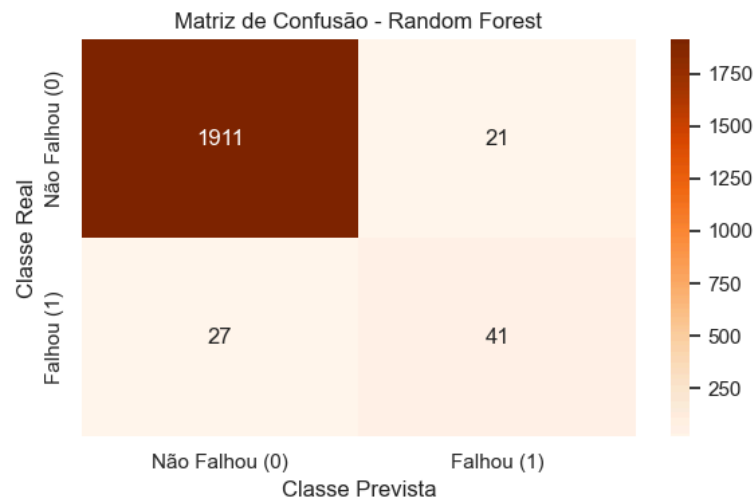


Figura 16 — Matriz de confusão do Random Forest.

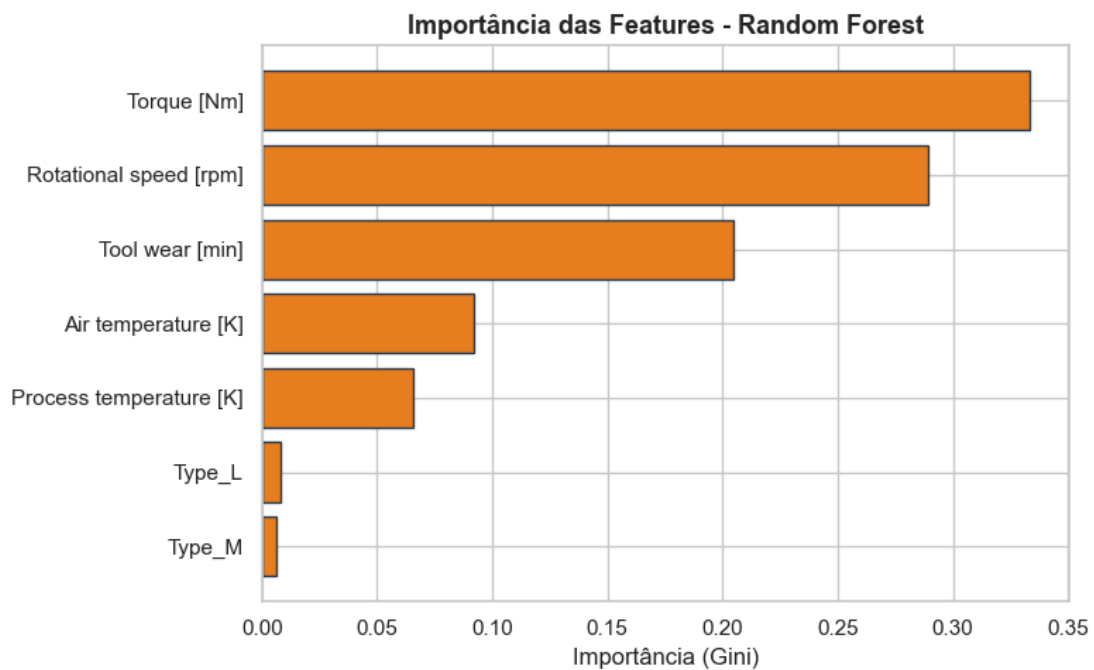


Figura 17 — Importância das features no Random Forest.

5.8 Árvore Podada vs. Random Forest

Métrica	Árvore Podada	Random Forest
Acurácia	0,9855	0,9760
Precisão (classe 1)	0,8000	0,6613
Recall (classe 1)	0,7647	0,6029
F1-Score (classe 1)	0,7820	0,6308
F2-Score (classe 1)	0,7715	0,6138

Resultado notável: a árvore individual podada superou o ensemble de Random Forest em todas as métricas — um achado discutido em detalhe na Seção 6.1.

6. Comparação e Escolha do Melhor Modelo

Para garantir uma comparação justa, todas as métricas dos três modelos (Rede Neural, Árvore de Decisão Podada e Random Forest) foram recalculadas no mesmo conjunto de teste, no mesmo momento. A Rede Neural utilizou o limiar de decisão calibrado ($\tau = 0,20$, determinado no conjunto de validação — Seção 4.7); os modelos baseados em árvore utilizaram seus limiares internos padrão.

Modelo	Acurácia	Precisão	Recall	F1-Score	F2-Score
Árvore de Decisão (podada)	0,9855	0,8000	0,7647	0,7820	0,7715
Rede Neural ($\tau = 0,20$)	0,9745	0,6269	0,6176	0,6222	0,6195
Random Forest	0,9760	0,6613	0,6029	0,6308	0,6138
Baseline (sempre 0)	0,9660	—	0,0000	—	0,0000

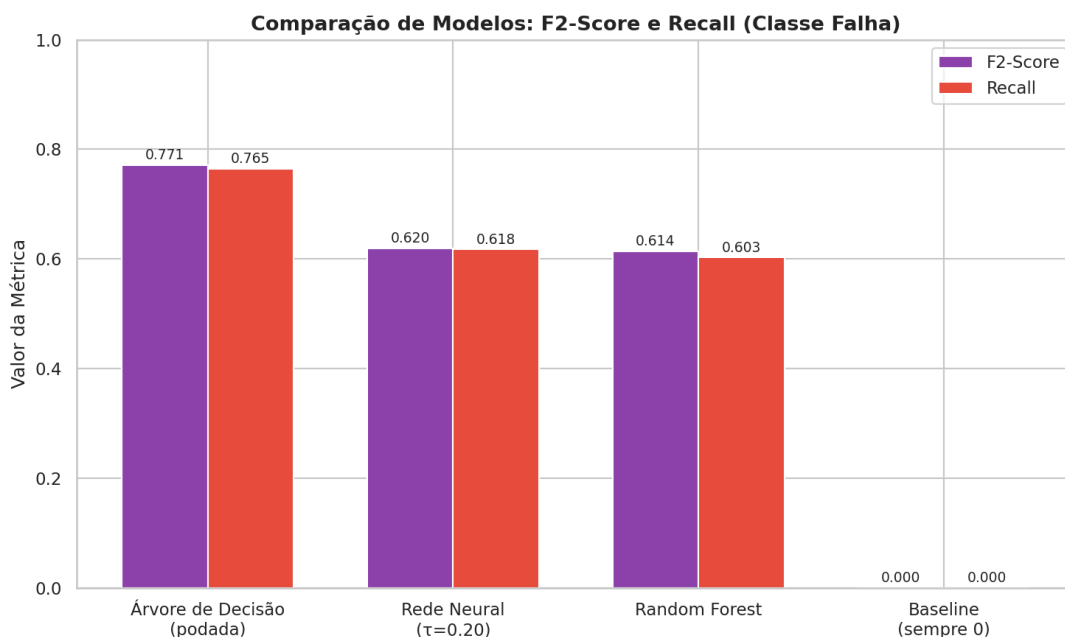


Figura 18 — Comparação de F2-Score e Recall entre os três modelos e o baseline.

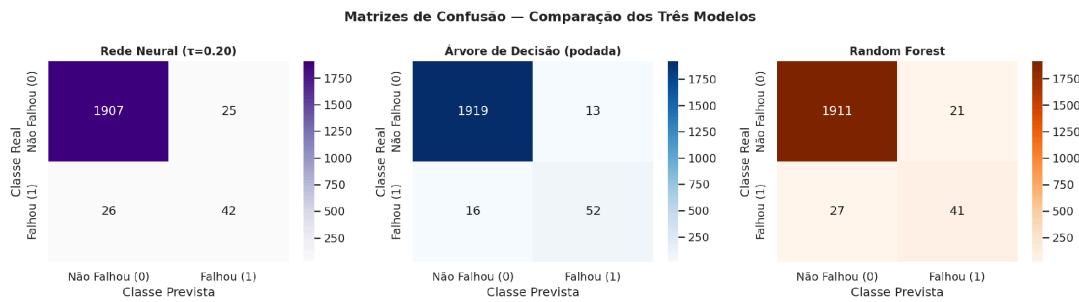


Figura 19 — Matrizes de confusão dos três modelos, lado a lado, no mesmo conjunto de teste.

6.1 Por que a Árvore de Decisão Superou o Random Forest?

Um resultado aparentemente contraintuitivo — a árvore individual podada superou o ensemble de 100 árvores em todas as métricas — encontra explicação na natureza estrutural do dataset AI4I 2020. As regras reais de falha são definidas por limiares rígidos entre poucos sensores: a falha de potência (PWF) depende de Torque $\times$ Velocidade Rotacional estar fora de uma faixa; a falha por desgaste (OSF) depende de Torque $\times$ Tool Wear exceder um limiar específico por tipo de produto. Esse padrão é exatamente o que uma árvore de decisão captura por natureza, já que seus nós são testes de limiar ($x \leq t$) que mapeiam diretamente para as regras físicas de falha.

O Random Forest introduz aleatoriedade — subamostragem de features em cada divisão — que, neste caso específico, prejudica mais do que ajuda: ao não ter acesso a todas as features simultaneamente em cada split, as árvores individuais do ensemble podem perder as combinações exatas que definem as falhas. O benefício clássico do Random Forest (redução de variância via agregação) é mais relevante em problemas com fronteiras de decisão complexas, ruidosas e de alta dimensionalidade — cenário distinto do presente dataset, com apenas 7 features e regras de falha bem definidas, onde a variância de uma única árvore bem podada já é baixa.

6.2 Convergência na Importância de Features

Tanto a Árvore de Decisão quanto o Random Forest convergiram em eleger essencialmente as mesmas variáveis como mais importantes: Torque, Tool wear, Rotational speed e Air temperature. Essa convergência não é coincidência — reflete as regras físicas reais de falha codificadas no dataset:

Modo de Falha	Regra Física	Features Envolvidas
PWF (Power Failure)	Potência = Torque $\times$ ω fora da faixa	Torque, Rotational speed
OSF (Overstrain Failure)	Torque $\times$ Tool wear excede limiar	Torque, Tool wear
HDF (Heat Dissipation)	$\Delta T < 8,6$ K e $\omega < 1380$ rpm	Air temp., Process temp., Rot. speed
TWF (Tool Wear Failure)	Tool wear entre 200–240 min	Tool wear

O fato de os modelos terem aprendido, a partir apenas dos dados, exatamente as variáveis que compõem as regras físicas do processo industrial reforça a confiança de que o sinal capturado é real, e não um artefato estatístico do conjunto de treino.

6.3 Vantagens Práticas do Modelo Escolhido

Além do desempenho quantitativo superior, a Árvore de Decisão oferece uma vantagem prática decisiva para o contexto industrial: interpretabilidade. A árvore pode ser visualizada e explicada diretamente a um engenheiro de manutenção — cada nó mostra qual sensor foi testado e qual limiar motivou a decisão, permitindo validar se a regra aprendida faz sentido físico. Em contraste, a Rede Neural é uma caixa-preta: seus 793 pesos distribuídos em duas camadas não oferecem explicação intuitiva sobre o motivo de uma predição específica. Para a adoção prática de um sistema de manutenção preditiva por uma equipe industrial, a confiança humana na ferramenta é um fator crítico, e a transparência da árvore de decisão facilita substancialmente essa adoção.

6.4 Modelo Final Escolhido

O modelo final escolhido para este projeto é a Árvore de Decisão Podada (Cost-Complexity Pruning), por apresentar o melhor F2-Score no conjunto de teste (0,7715), o melhor equilíbrio entre Precisão (0,80) e Recall (0,76), e por ser o modelo mais interpretável entre os avaliados — alinhado tanto à natureza do problema quanto aos requisitos práticos de adoção em ambiente industrial.

7. Conclusão

7.1 Pergunta de Pesquisa

É possível prever falhas em máquinas industriais a partir de leituras de sensores operacionais (temperatura, velocidade rotacional, torque, desgaste da ferramenta), e qual modelo de Aprendizagem de Máquina é mais adequado para essa tarefa?

7.2 Resultado Obtido

Sim, é possível. Dentre os três modelos avaliados — Rede Neural (MLP), Árvore de Decisão e Random Forest —, a Árvore de Decisão Podada alcançou o melhor desempenho na detecção de falhas, combinando alto Recall (76,5%) com boa Precisão (80,0%), resultando no maior F2-Score (0,7715) no conjunto de teste, muito acima do baseline de referência.

7.3 Principais Achados

- **Importância de features consistente com a física do processo:** todos os modelos convergiram em identificar Torque, Tool wear e Rotational speed como as variáveis mais relevantes — exatamente as que compõem as regras físicas reais de falha do maquinário (Power Failure, Overstrain Failure, Heat Dissipation Failure), confirmando que os modelos capturaram sinal real.
- **A calibração do limiar de decisão é essencial em problemas desbalanceados:** a Rede Neural com limiar padrão (0,5) apresentou desempenho inferior ao seu potencial real ($F2 = 0,5128$ no teste); calibrando o limiar para 0,20 — escolhido exclusivamente a partir do conjunto de validação, preservando a integridade do teste — o F2-Score no teste subiu para 0,6195, demonstrando que, em cenários de desbalanceamento severo e custo assimétrico entre erros, o limiar de decisão é um hiperparâmetro que não deve ser ignorado, mas que precisa ser calibrado com o devido cuidado metodológico.

- **A árvore individual superou o ensemble:** a Árvore de Decisão Podada superou o Random Forest em todas as métricas, resultado explicado pela natureza do problema — falhas definidas por regras de limiar simples entre poucos sensores, cenário no qual uma única árvore bem podada já captura toda a estrutura relevante, e a aleatoriedade introduzida pelo ensemble se torna ruído.
- **Interpretabilidade como diferencial prático:** a Árvore de Decisão não apenas obteve o melhor desempenho quantitativo, mas também oferece interpretabilidade total — cada decisão pode ser explicada e validada por um engenheiro de manutenção, o que facilita a confiança e a adoção prática do sistema em ambiente industrial.

7.4 Limitações

- **Natureza sintética do dataset:** o AII 2020 é um dataset simulado, gerado com regras predefinidas que emulam padrões industriais reais. Os resultados validam a abordagem metodológica empregada, mas a transferência para dados de campo reais exigiria recalibração e validação adicional em ambiente de produção.
- **Escopo do maquinário:** o dataset modela um único tipo genérico de maquinário industrial. A validade dos modelos e das features selecionadas está circunscrita a esse contexto específico, não devendo ser generalizada sem adaptação a outros tipos de equipamento.

7.5 Considerações Finais

Este projeto demonstrou, de ponta a ponta, a aplicação dos conceitos de Aprendizagem de Máquina estudados na disciplina — desde o dimensionamento teórico de uma Rede Neural pela Dimensão VC e pela Regra de Ouro da Generalização, passando pela poda de Árvores de Decisão via Minimal Cost-Complexity Pruning com validação cruzada, até a calibração criteriosa de métricas e limiares de decisão adequados a um problema real de desbalanceamento severo de classes. O resultado final — um modelo interpretável, fisicamente coerente e com desempenho robusto na detecção de falhas — ilustra o valor prático da manutenção preditiva como aplicação de Aprendizagem de Máquina a um problema genuíno enfrentado pela indústria.